Лабораторная работа №2

REST API with Amazon SQS

Студент: Абдрахманова Адель

1. Цель работы

Цель лабораторной работы — реализовать REST API на Python с использованием Flask, научиться работать с HTTP-запросами и JSON, а также реализовать асинхронную обработку задач с помощью очереди Amazon SQS.

В отличие от лабораторной работы №1, где использовались TCP sockets и собственный протокол с командами ECHO, TIME и QUIT, в этой работе используется стандартный HTTP и REST API.

Дополнительно была реализована очередь задач. API принимает задачу и передаёт её в Amazon SQS, после чего отдельная программа worker.py получает задачу, обрабатывает её и сохраняет результат.

2. Краткое описание проекта

Проект состоит из REST API, очереди Amazon SQS и отдельного worker.

Основное Flask-приложение находится в файле app.py.

Оно поддерживает следующие запросы:

- POST /echo — возвращает переданное клиентом сообщение;
- GET /time — возвращает текущее время сервера;
- POST /jobs — создаёт новую задачу и отправляет её в Amazon SQS;
- GET /jobs/<job_id> — возвращает текущий статус задачи и результат после её выполнения.

Для каждой новой задачи создаётся уникальный идентификатор UUID.

Например:

4360de71-b0c5-4792-8ec7-9dbbeda4075e

После получения новой задачи API отправляет её в Amazon SQS и сразу возвращает клиенту статус queued и HTTP-код 202 Accepted.

Обработка задачи выполняется отдельно в worker.py.

Worker получает сообщение из Amazon SQS, имитирует длительную работу с помощью задержки в две секунды, переводит полученный текст в верхний регистр и сохраняет результат.

Общая архитектура проекта:

Client

  |

  | HTTP request

  v

Flask REST API

  |

  | POST /jobs

  v

Amazon SQS

```
  |
  v
worker.py
  |
  | processing
  v
results/<job_id>.json
```

3. Использованные технологии

В проекте использовались:

- Python 3;
- Flask;
- HTTP;
- JSON;
- Amazon SQS;
- Boto3;
- UUID;
- Git и GitHub.

Flask используется для создания REST API.

Boto3 используется для взаимодействия Python-приложения с AWS и Amazon SQS.

Amazon SQS используется как очередь сообщений между API и worker.

AWS credentials не хранятся внутри исходного кода и не добавляются в Git-репозиторий.

4. Реализованные функции

4.1 Endpoint /echo

Endpoint:

POST /echo

принимает JSON:

```
{
  "message": "hello"
}
```

и возвращает:

```
{
  "reply": "hello"
}
```

Если поле message отсутствует или имеет неправильный формат, сервер возвращает HTTP-код:

400 Bad Request.

4.2 Endpoint /time

Endpoint:

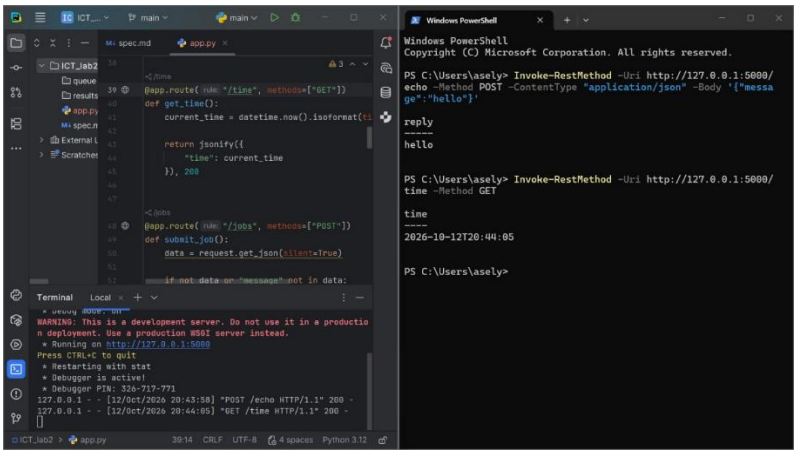
GET /time

возвращает текущую дату и время сервера.

Пример ответа:

```
{
  "time": "2026-10-20T15:00:00"
}
```

Скриншот результата /time:



4.3 Обработка ошибок

REST API проверяет входные данные и корректно обрабатывает ошибки.

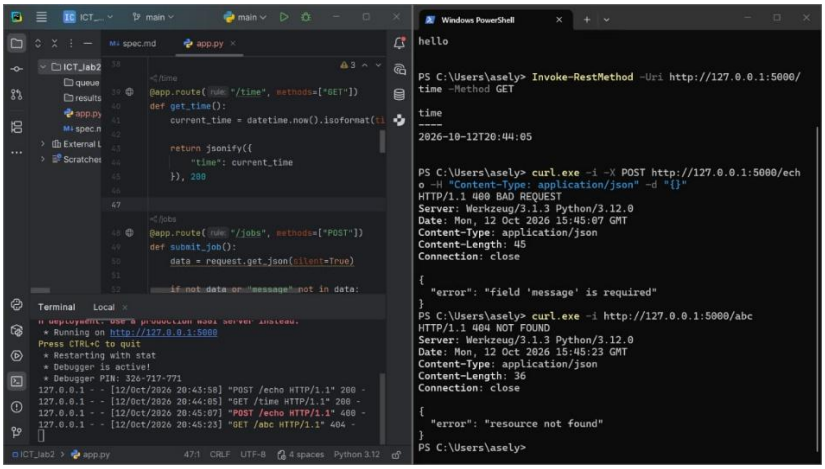
Если клиент отправляет неправильные данные, сервер возвращает:

400 Bad Request.

Если клиент обращается к несуществующему endpoint или несуществующей задаче, сервер возвращает:

404 Not Found.

Скриншот ошибок:



4.4 Создание новой задачи

Endpoint:

POST /jobs

принимает сообщение:

{
  "message": "hello from lab 2"
}

После этого сервер создаёт уникальный job_id и отправляет задачу в Amazon SQS.

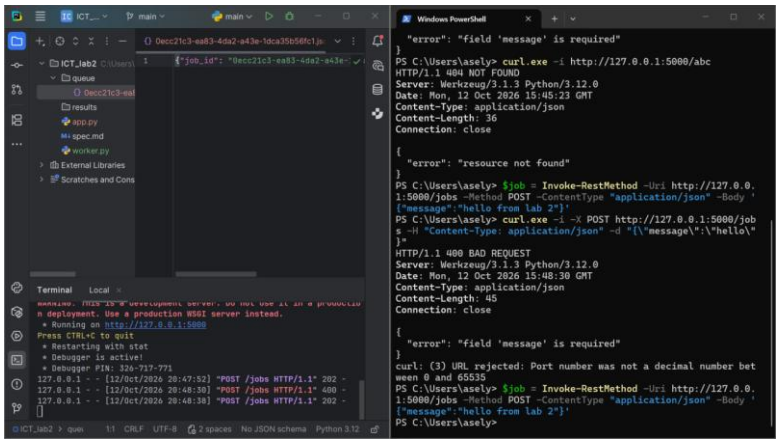
API не ждёт выполнения задачи и сразу возвращает:

{
  "job_id": "UUID",
  "status": "queued"
}

с HTTP-кодом:

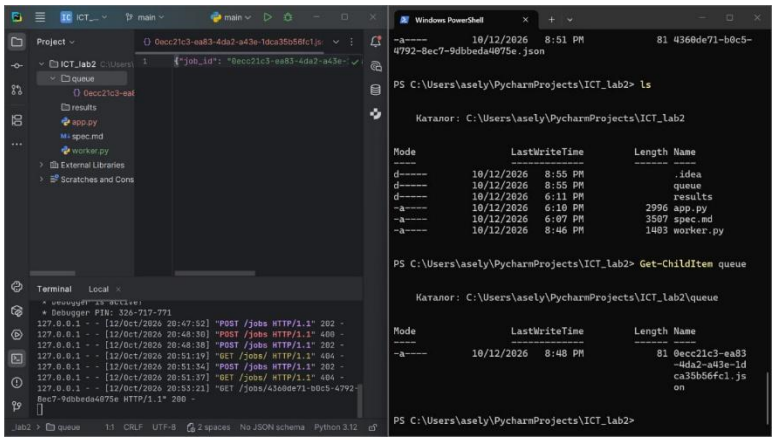
202 Accepted.

Скриншот создания задачи:



4.5 QUEUE

Скриншот:



4.6 Worker

worker.py работает отдельно от Flask API.

Последовательность работы worker:

получить сообщение из SQS

↓

прочитать job_id и message

↓

подождать 2 секунды

↓

обработать сообщение

↓

сохранить результат

↓

удалить сообщение из SQS

В данной лабораторной обработка заключается в переводе текста в верхний регистр.

Например:

hello from lab 2

становится:

HELLO FROM LAB 2

Сообщение удаляется из Amazon SQS только после успешной обработки.

4.7 Получение результата

Endpoint:

GET /jobs/<job_id>

позволяет проверить состояние задачи.

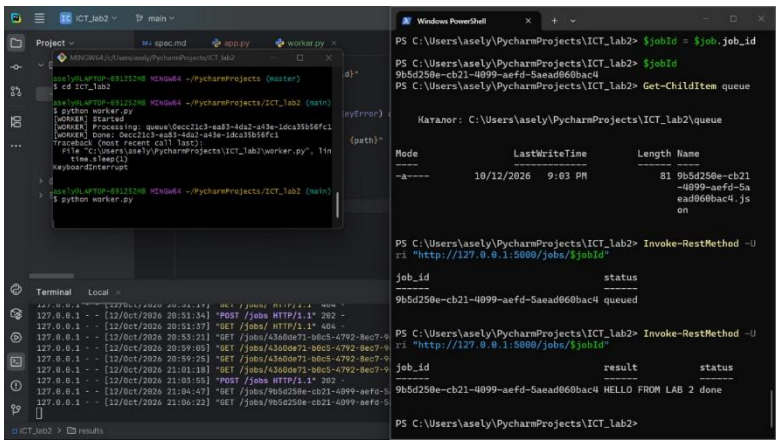
Пока задача ожидает обработки:

```
{
  "job_id": "UUID",
  "status": "queued"
}
```

После завершения:

```
{
  "job_id": "UUID",
  "status": "done",
  "result": "HELLO FROM LAB 2"
}
```

Скриншот выполненной задачи:



5. Ответы на вопросы

5.1 Как REST принципиально отличается от socket protocol из Lab 1? Что даёт то, что каждый запрос self-contained?

В лабораторной работе №1 использовалось прямое TCP-соединение между клиентом и сервером.

Клиент подключался к серверу через socket и сохранял соединение открытым до отправки команды QUIT.

Также использовался собственный простой протокол:

ECHO

TIME

QUIT

В лабораторной работе №2 используется стандартный HTTP и REST API.

REST-запрос содержит всю информацию, которая нужна серверу для его обработки: HTTP-метод, URL и при необходимости JSON-данные.

Каждый запрос является self-contained, то есть самостоятельным.

Это означает, что серверу не нужно помнить предыдущие запросы клиента.

Такой подход делает систему проще и удобнее для масштабирования. Можно запустить несколько экземпляров сервера, и разные запросы одного клиента могут обрабатываться разными экземплярами приложения.

5.2 Почему endpoint отправки задачи возвращает 202, а не 200?

Когда клиент отправляет запрос:

POST /jobs

задача ещё не выполнена.

API только принимает её, создаёт job_id и отправляет сообщение в Amazon SQS.

Саму работу позже выполняет worker.py.

Поэтому используется HTTP-код:

202 Accepted.

Он означает, что запрос успешно принят для дальнейшей обработки, но конечный результат ещё не готов.

Код 200 OK больше подходит для ситуации, когда запрос уже полностью выполнен.

5.3 Зачем нужна очередь, если задачу можно выполнить прямо внутри HTTP-запроса?

Для очень быстрых операций задачу действительно можно выполнить непосредственно внутри HTTP-запроса.

Но некоторые операции могут занимать много времени.

Например:

- обработка видео или изображений;
- Machine Learning;
- создание больших отчётов;
- обработка файлов;
- сложные вычисления;
- массовая отправка сообщений.

Если выполнять такую работу внутри HTTP-запроса, клиенту придётся ждать всё время выполнения.

При использовании очереди API быстро принимает задачу, отправляет её в Amazon SQS и возвращает клиенту 202 Accepted.

После этого отдельный worker выполняет задачу независимо от HTTP-запроса.

Это уменьшает время ожидания клиента и позволяет системе постепенно обрабатывать большое количество задач.

5.4 Что произойдёт, если worker упадёт после получения сообщения, но до удаления его из очереди?

Когда worker получает сообщение из Amazon SQS, оно не удаляется сразу.

На определённое время сообщение становится невидимым для других workers. Это называется Visibility Timeout.

Worker должен сначала обработать задачу и сохранить результат.

Только после успешной обработки сообщение удаляется из SQS.

Если worker упадёт до удаления сообщения, после окончания Visibility Timeout сообщение снова станет доступным в очереди.

Другой worker или тот же worker после перезапуска сможет получить его и повторить обработку.

Таким образом, задача не будет потеряна.

При этом существует возможность повторной обработки одной и той же задачи, поэтому реальные распределённые системы обычно проектируют так, чтобы повторное выполнение операции не приводило к неправильному состоянию данных.

6. Git и спецификация

Перед написанием основного кода была создана спецификация spec.md.

В ней были определены:

- основные endpoints;
- входные и выходные данные;
- HTTP-коды;
- возможные ошибки;
- ограничения;
- assumptions;
- non-functional requirements;
- использование Amazon SQS.

Начальная версия спецификации была зафиксирована в Git до реализации основной части проекта.

После этого были реализованы app.py и worker.py.

Такой подход позволяет сначала определить требования к системе, а затем реализовать их в коде.

7. Вывод

В ходе лабораторной работы был реализован REST API на Flask и изучена работа с HTTP-запросами и JSON.

Были реализованы endpoints для возврата сообщения и времени, обработка ошибок и асинхронное выполнение задач.

Для очереди сообщений использовался Amazon SQS.

Отдельный worker получает задачи из очереди, обрабатывает их и сохраняет результат.

Главное отличие от предыдущей лабораторной работы заключается в переходе от постоянного TCP-соединения и собственного socket-протокола к REST API, где каждый HTTP-запрос является независимым.

Также была изучена разница между синхронной и асинхронной обработкой и показано, зачем в распределённых системах используются очереди сообщений.